

BERT_BART_ELECTRA_T5 모델

BERT

- BERT (Bidirectional Encoder Representations from Transformers)는 2018년 Google에서 발표한 자연어 처리(NLP) 모델로, 트랜스포머 인코더의 양방향 구조를 기반

- 양방향 인코딩 (Bidirectional Encoding)**

- 기존 모델은 왼쪽 → 오른쪽 방향(또는 반대)으로 단어를 순차적으로 처리함
- BERT는 양쪽 방향(좌우 모두)의 문맥을 고려해 각 단어의 의미를 더 정확히 파악함
- 결과적으로: 문장의 의미를 더 잘 이해할 수 있으며, 다양한 자연어 처리 작업에서 기존 모델 대비 높은 성능을 보임

- 사전 학습 특징**

- 대규모 말뭉치를 기반으로 사전 학습(pretraining)됨 → 이후 다양한 작업에 대해 전이 학습(fine-tuning) 가능
- 장점: 사전 학습으로 높은 정확도와 적은 연산 자원으로 빠른 적용 가능

구조적 특징

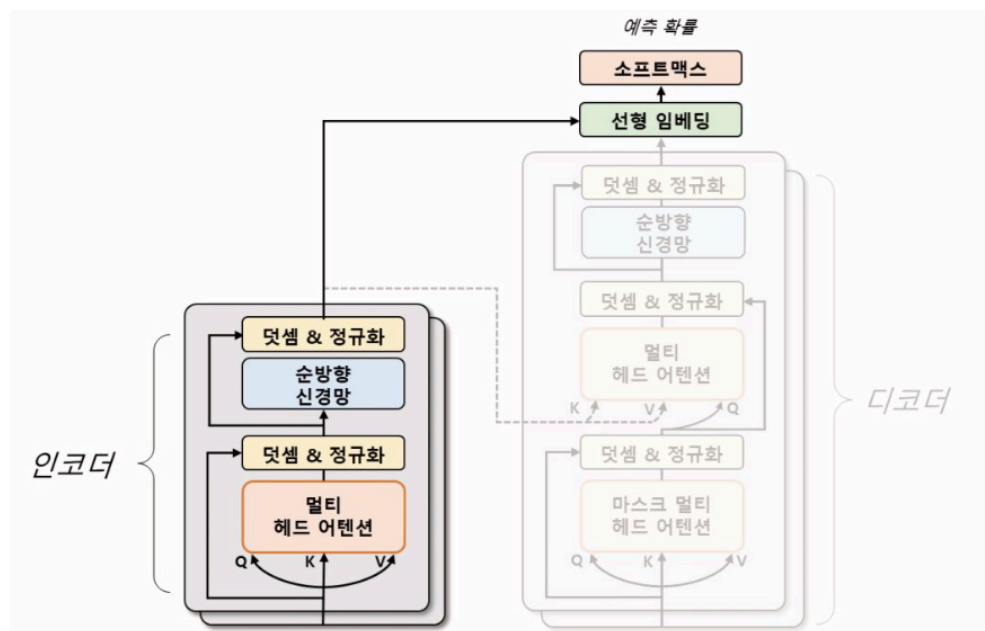


그림 7.13 트랜스포머와 BERT 모델 비교

- BERT는 트랜스포머의 인코더 블록만 사용 (디코더 없음)

- 주요 구성:

- 단어 임베딩 (Token Embedding)
- 위치 임베딩 (Position Embedding)
- 트랜스포머 인코더 계층

- 학습 방법 : BERT는 두 가지 사전 학습 기법을 사용

- MLM (Masked Language Modeling)** : 입력 문장에서 일부 단어를 [MASK] 로 가리고 해당 단어를 예측
- NSP (Next Sentence Prediction)** : 두 문장이 연속된 문장인지 여부를 예측

사전 학습 방법

1. 마스킹된 언어 모델링 (Masked Language Modeling, MLM)

- 입력 문장에서 일부 단어를 마스킹하고, 해당 단어를 예측하는 방식
- 예시: "I'm learning PyTorch" → "I'm [MASK] PyTorch"
- BERT는 [MASK] 에 들어갈 단어를 양방향 문맥을 참고해 예측함

- 입력 전체 문맥을 고려하므로, 문장 전체 의미 파악 능력이 향상됨

2. 다음 문장 예측 (Next Sentence Prediction, NSP)

- 두 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 실제로 이어지는지를 판별
- 예시:
 - 문장1: "I'm learning PyTorch"
 - 문장2: "PyTorch is a machine learning library"
 - BERT는 이 두 문장이 연속 문장인지 여부를 판단함
- 문장 간의 의미적 관계 및 연속성 파악 능력을 학습

3. 특수 토큰

- BERT는 모델이 문장을 이해하고 구분할 수 있도록 특수 토큰들을 추가
 - **[CLS]**: 입력 문장의 시작에 삽입되는 토큰
 - 전체 문장을 대표하는 벡터 역할
 - 문장 분류, 감정 분석 등에서 문장 전체의 의미를 나타냄
 - **[SEP]**: 두 문장을 구분하기 위해 사용되는 토큰
 - NSP나 다중 문장 입력에서 문장 경계를 표시함
 - **[MASK]**: MLM에서 마스킹할 단어 자리에 삽입하는 토큰
 - 선택된 단어를 가리고 모델이 그 단어를 예측하게 함

BERT 모델의 입력 임베딩과 손실함수

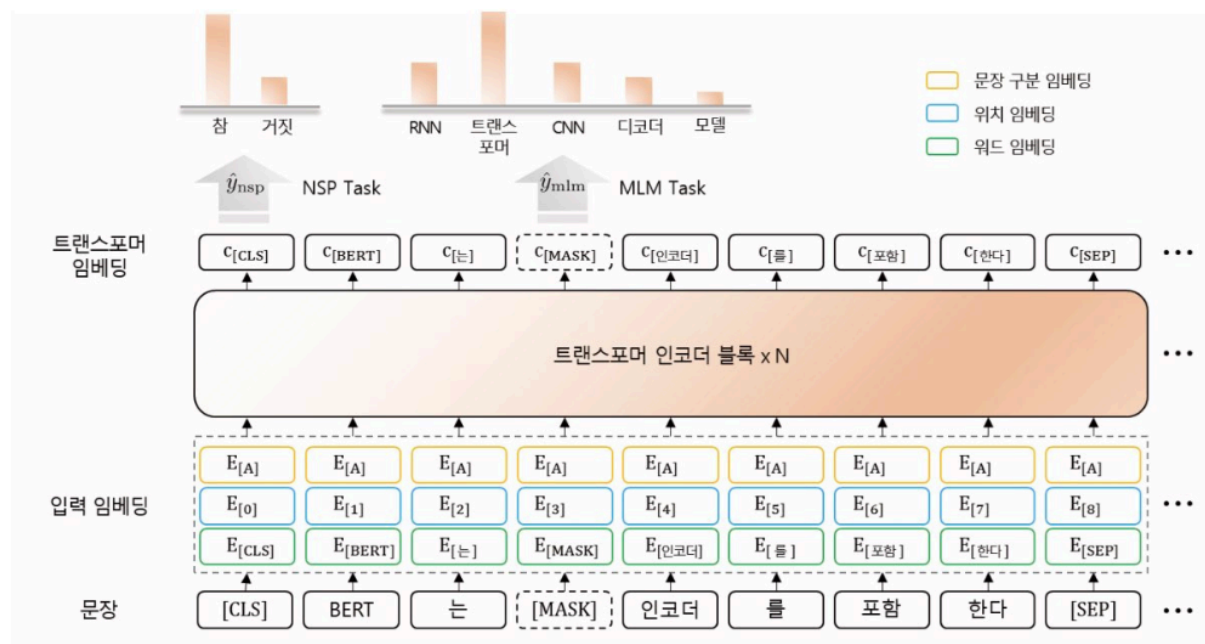


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

• 입력 시퀀스 구성

- BERT 모델의 입력 시퀀스는 다음과 같은 구조를 가짐: **[CLS] 문장-1 [SEP] 문장-2 [MASK] [SEP]**
 - 첫 문장과 두 번째 문장을 **[SEP]**로 구분하고, **[CLS]**는 문장 전체 의미를 표현하며, 필요한 위치에 **[MASK]**를 삽입

• [MASK] 적용 규칙

- 전체 토큰 중 약 15%를 마스킹 대상으로 선택하며, 다음과 같이 처리:
 - 80%는 **[MASK]** 토큰으로 대체
 - 10%는 랜덤 단어로 대체
 - 10%는 원래 단어 그대로 유지
- 모델이 단순히 **[MASK]**만 학습하는 것이 아니라, 다양한 상황에서도 유연하게 예측할 수 있도록 설계됨

- 사전 학습 이후 활용

- 사전 학습을 마친 BERT는 미세 조정(fine-tuning)을 통해 다양한 자연어 처리(NLP) 작업에 활용 가능
 - 예: 문장 분류, 감정 분석, 질문 응답, 기계 번역 등
- 작업에 맞는 출력 계층과 손실 함수만 추가하면 다양한 태스크에 적용 가능
- 분류 작업: 일반적으로 [CLS] 토큰 벡터를 사용해 전체 문장 분류 수행
- 토큰별 예측: 토큰 단위의 예측이 필요한 경우, 모든 토큰 벡터를 사용

BART

- BART (Bidirectional Auto-Regressive Transformer)

- 2019년 메타(Facebook AI Research)에서 발표한 트랜스포머 기반 자연어 처리 모델
- BERT의 인코더와 GPT의 디코더를 결합한 Seq2Seq (Sequence-to-Sequence) 구조
- 노이즈 제거 오토인코더(Denoising Autoencoder) 방식으로 사전 학습 수행

BART의 사전 학습 방식

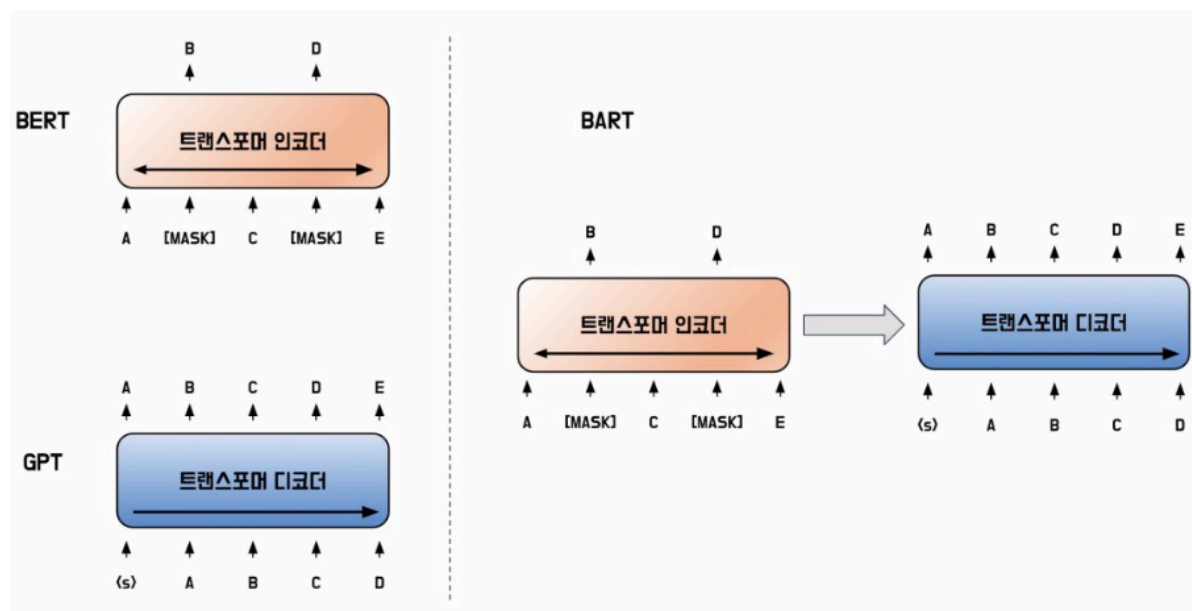


그림 7.15 BART 사전 학습 시각화

- 입력 문장에 노이즈를 추가한 후, 이를 기반으로 원래 문장을 복원하도록 학습
- 노이즈 방식 예시 : BERT처럼 일부 단어를 무작위로 마스킹, GPT처럼 문장의 일부를 삭제하거나 섞음
- 학습 구조
 - 노이즈가 추가된 문장 → 인코더 입력
 - 정상 문장 → 디코더 출력으로 예측
 - 즉, 입력: 손상된 문장 → 출력: 복원된 원문

BERT 모델 구조

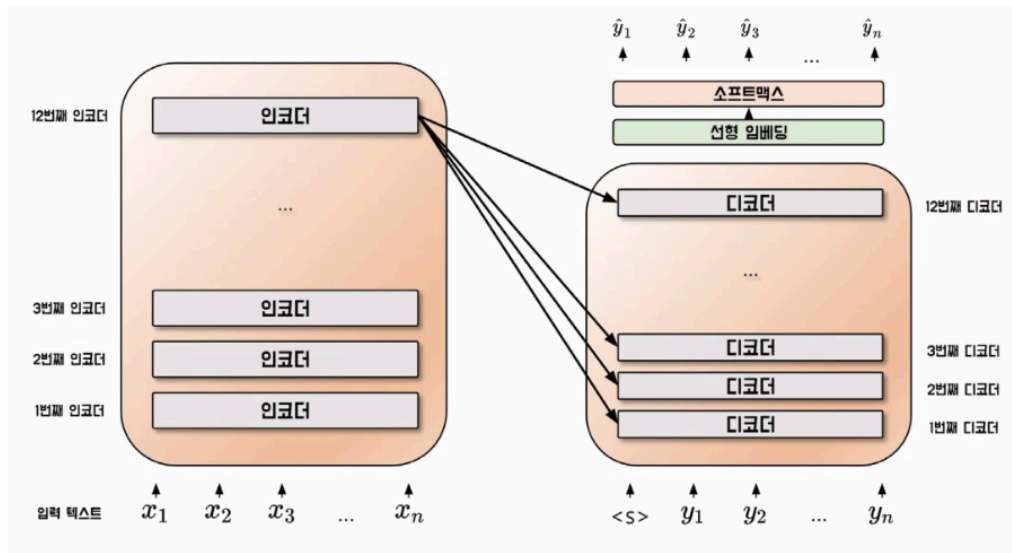


그림 7.16 BART 모델 구조

• BERT vs GPT vs BART

모델	구조	특징
BERT	인코더만 사용	양방향 문맥 이해, 마스킹된 단어 예측 (MLM)
GPT	디코더만 사용	왼쪽부터 순차적 예측 (Auto-regressive)
BART	인코더 + 디코더	BERT + GPT 결합, 복원 중심 학습, 문장 생성/변환에 강함

- BART는 BERT처럼 입력 전체 문맥을 인식하고, GPT처럼 생성 능력도 가짐

• BART의 구조적 장점

- 입력 구조와 의미를 보존하면서 다양한 변형을 학습
- GPT의 한계였던 순방향 정보만 처리 가능 문제를 극복
- BERT의 한계였던 생성 불가능 문제도 개선

• 트랜스포머와의 차이점

항목	트랜스포머	BART
어텐션 적용 범위	인코더 모든 계층 ↔ 디코더 모든 계층	인코더 마지막 계층 ↔ 디코더 모든 계층
메모리 사용 최적화	없음	어텐션 범위를 제한하여 효율화

- BART는 인코더 마지막 계층과 디코더 각 계층 사이에만 어텐션 연산을 수행함으로써 정보 전달 최적화 및 연산량 절감 가능

• 문장 표현 생성 방식

- 인코더는 입력 문장의 각 단어를 임베딩하고 여러 층을 통과해 가장 잘 문장을 대표하는 벡터 생성
- 디코더는 해당 벡터 + 자신이 예측한 이전 단어들을 참고하여 출력 문장 생성

• BART의 활용

- 문장 생성, 기계 번역, 문서 요약, 질문 응답 등 생성 기반 NLP 작업에 최적화
- BERT와 달리 텍스트를 입력받고 다시 텍스트를 출력하는 태스크에 효과적

사전 학습 방법

- BART는 BERT의 Masked Language Modeling(MLM) 기법 외에도 다양한 노이즈 삽입 기법을 활용하여 입력 문장을 변형시키고, 이를 복원하는 방식으로 학습됨
 - → 이 과정은 노이즈 제거 오토인코더(Denoising Autoencoder) 구조에 기반

노이즈 기법

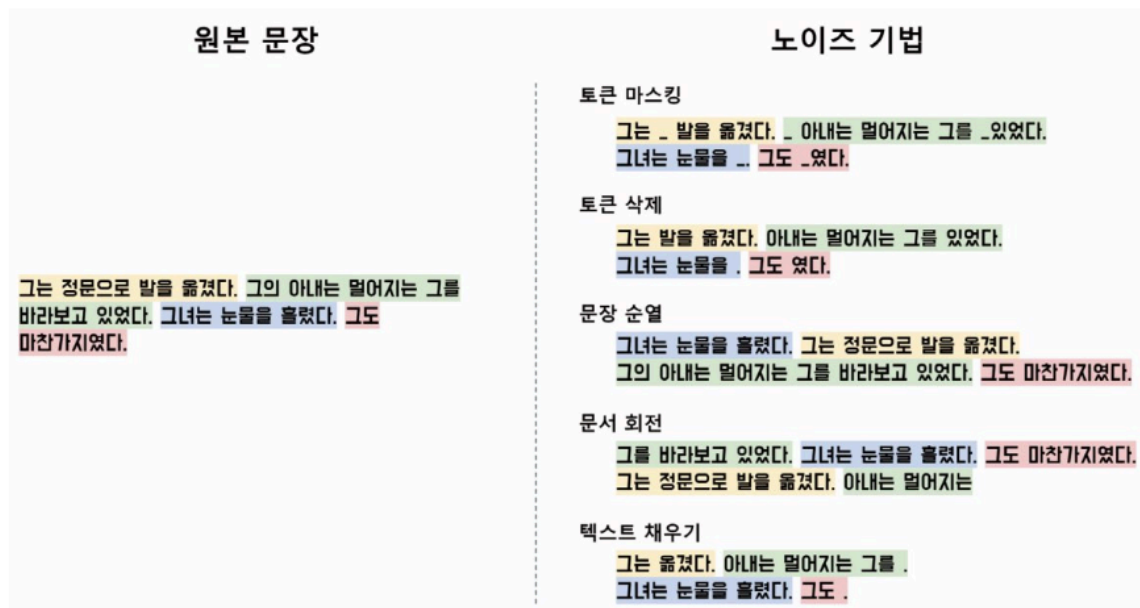


그림 7.17 BART 노이즈 기법

1. 토큰 마스킹 (Token Masking)

- BERT의 MLM과 동일한 방식
- 입력 문장의 일부 토큰을 [MASK] 로 치환 → 모델은 마스킹된 위치의 원래 단어를 예측함
- 효과: 문장 내에서 중요한 단어와 문맥 정보를 학습

2. 토큰 삭제 (Token Deletion)

- 특정 토큰을 마스킹하는 것이 아니라 완전히 삭제
- 입력에서 제거된 토큰의 위치를 모델이 추정해야 함
- 효과: 불필요한 정보를 필터링하고 예측 능력과 일반화 성능 향상

3. 문장 순열 (Sentence Permutation)

- 문장의 끝에 있는 마침표(.)를 기준으로 문장을 나눈 뒤, 문장 순서를 섞음
- 효과: 모델이 문장 내 의미 연결성을 더 정확히 파악하게 함 → 순서가 바뀌어도 의미를 유지하며 처리 가능

4. 문서 회전 (Document Rotation)

- 임의의 토큰에서 문장이 시작되도록 함 (시작 지점을 이동) → 문장 순서는 그대로 유지됨
- 효과: 문서의 시작점이 어디인지 인식하도록 도와줌

5. 텍스트 채우기 (Text Infilling)

- 여러 연속된 토큰을 하나의 스패(span) 으로 묶고 해당 구간을 [MASK] 로 대체
- 마스킹 구간의 길이는 포아송 분포(Poisson distribution)를 사용해 무작위로 설정
 - 예: 평균 $\lambda=3$ 인 포아송 분포
 - 길이가 0이면 토큰 삭제와 동일한 효과
- 효과: 연속된 단어 예측 능력 강화 → 문맥 내 결손된 정보를 유추하여 문장을 더욱 정밀하게 복원

이러한 다양한 노이즈 기법을 통해

→ BART는 단순한 단어 예측을 넘어 문장 및 문서 구조 전체를 이해하고 복원하는 능력을 습득함

→ 결과적으로 BART는 문장 생성, 요약, 번역, 질문 응답 등 생성 기반 NLP 작업에서 매우 높은 성능

미세 조정 방법

- BART는 인코더와 디코더를 모두 사용하는 구조이기 때문에, 다운스트림 작업에 맞춰 입력 문장을 구성해야 함
- 즉, 인코더 입력과 디코더 입력이 각각 다르게 구성될 수 있음

미세 조정 구조

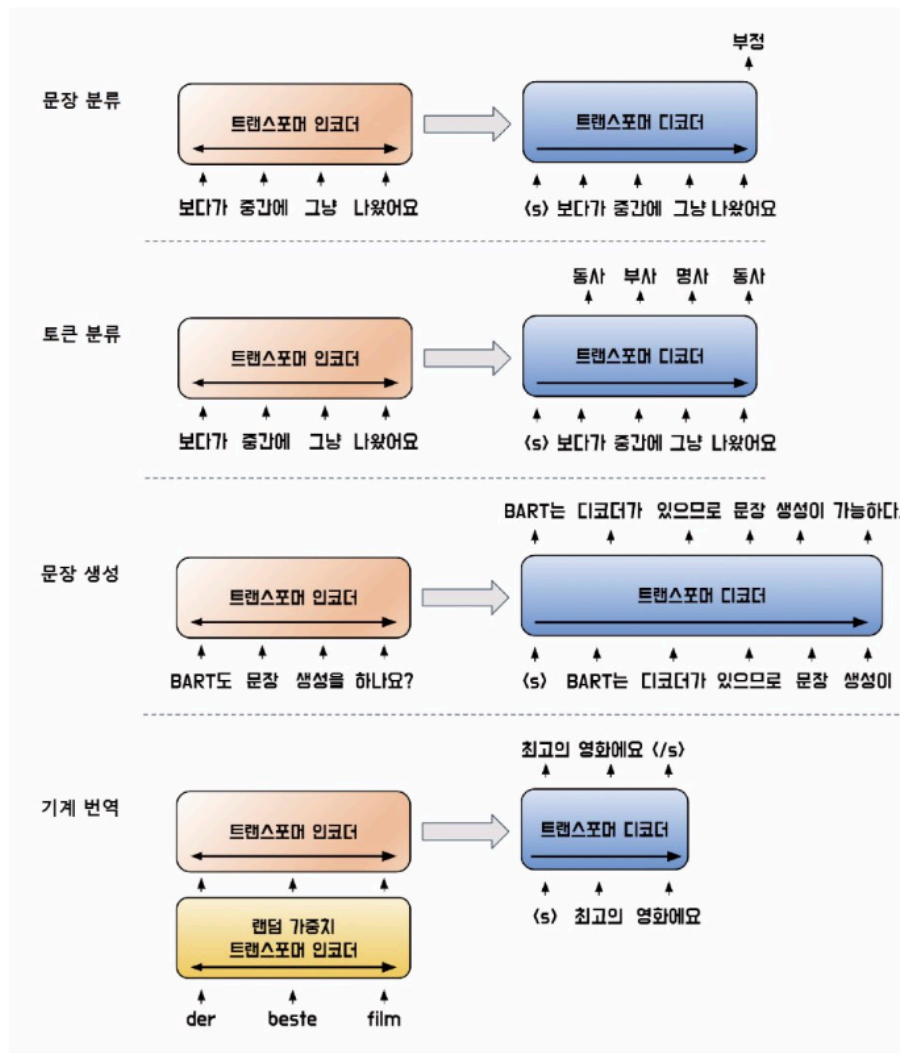


그림 7.18 BART 미세 조정 구조

1. 문장 분류

- 입력 문장을 인코더와 디코더에 동일하게 입력 → 디코더의 마지막 토큰 은닉 상태를 분류기의 입력으로 사용
- BERT의 [CLS] 토큰과 유사한 역할을 수행하지만, BART는 전체 입력과 어텐션이 적용된 은닉 상태 전체를 활용함

2. 토큰 분류

- 문장 분류와 동일하게 입력을 인코더와 디코더에 동일하게 전달
- 디코더의 각 시점별 은닉 상태를 이용해 각 토큰별 분류 수행
- 디코더는 전체 입력에 대해 시점별 어텐션 연산을 모두 수행

3. 문장 생성 기반 작업

- BART는 디코더를 사용하므로, BERT가 처리하지 못했던 텍스트 생성 작업까지 수행 가능
- 예: 추상적 질문 응답 (Abstractive Question Answering), 문장 요약 (Summarization)

- BERT는 출력 생성을 할 수 없지만, BART는 출력까지 생성할 수 있음

• BART의 미세 조정 학습 방식

- BART의 사전 학습 방식이 **노이즈 입력 → 정답 복원** 방식이므로, **기계 번역** 등과 유사한 방식으로 새로운 다운스트림 작업에 적용 가능

• 기계 번역 예시

- 기존 문장에 기계 번역용 인코더를 추가
- 첫 번째 인코더는 단어 시퀀스를 그대로 사용하지 않음
- 디코더는 사전 학습된 벡터와 단어 시퀀스를 사용해 번역을 생성함

• 학습 단계 구성

- [1단계] 새로운 트랜스포머 인코더를 추가 → 해당 인코더는 위치 임베딩과 함께 동작 → **첫 번째 인코더 층의 셀프 어텐션 입력 행렬 가중치만** 학습함
- [2단계] **모든 신경망 가중치**를 학습 대상으로 포함해 **전면 재학습**

ELECTRA

- ELECTRA: Efficiently Learning an Encoder that Classifies Token Replacements Accurately
- 2020년 Google에서 발표된 트랜스포머 기반 사전 학습 언어 모델
- **주요 특징**
 - BERT와 달리 [MASK]를 사용하지 않음
 - 생성자(Generator) + 판별자(Discriminator) 구조로 구성
 - GAN(Generative Adversarial Network) 유사한 훈련 구조

- **ELECTRA vs BERT**

항목	BERT	ELECTRA
방식	마스킹된 단어 예측 (MLM)	바뀐 단어 판별 (RTD)
입력	일부 단어를 [MASK] 로 바꿈	생성된 단어로 바꿈
목표	[MASK] 에 원래 단어를 채움	바뀐 단어인지 아닌지를 구분
훈련 효율	느림 (많은 마스킹 필요)	빠름 (모든 토큰 사용)

사전 학습 방법

- 생성자 (Generator)
 - 입력 문장의 15%를 마스킹 → 마스킹된 토큰을 예측하는 BERT 기반 MLM 모델 → 원래 토큰과 비슷하지만 다르게 예측된 토큰으로 바꿈
- 판별자 (Discriminator)
 - 변형된 문장을 입력받고, 각 토큰이 원래 토큰인지 생성된 토큰인지 구별
 - 이를 RTD (Replaced Token Detection) 작업이라고 부름

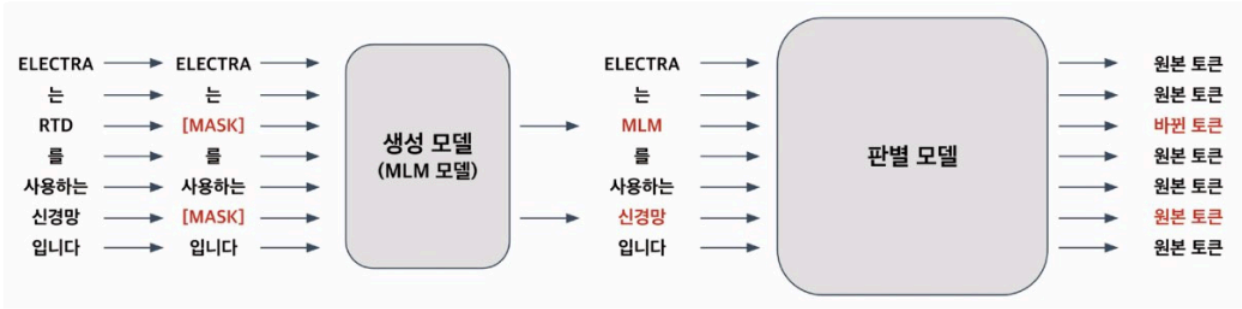


그림 7.20 ELECTRA 모델 학습 방법

- 생성자는 마스킹된 토큰을 복원 (BERT와 동일)
- 판별자는 바뀐 토큰인지 아닌지를 맞힘

GAN과의 차이

- **GAN:** 생성자가 만든 토큰을 진짜처럼 보이게 학습
- **ELECTRA**
 - GAN과 유사하지만, 완전한 노이즈를 입력으로 쓰지 않고 마스킹된 문장을 입력으로 받음
 - 판별자는 변형된 문장을 보고 각 토큰이 진짜인지 판별
 - **ELECTRA의 장점**
 - 더 적은 연산량으로도 빠르고 효율적으로 학습 가능
 - 모든 입력 토큰을 학습에 활용 가능 (BERT는 마스킹된 일부만 사용)
 - 모델 크기가 작아도 높은 성능 발휘 가능

- 생성자와 판별자는 모두 트랜스포머 인코더 구조를 따름

- 두 모델이 같은 계층 수로 구성되면 가중치 공유 가능, 학습 속도 향상
- [그러나 문제 발생] 생성자의 성능이 너무 높아지면 판별자가 학습할 수 없음
- [해결책]
 - 생성자의 크기를 판별자의 1/2 또는 1/4로 축소
 - 입력 임베딩 계층만 가중치 공유, 나머지 계층은 분리
 - → 성능 밸런스를 맞추며 효율적 학습 가능
- 사전 학습 이후 활용 방식
 - ELECTRA는 사전 학습이 끝나면 생성자 사용 중단 → 오직 판별자만 사용하여 다운스트림 작업 수행
 - 판별자는 BERT처럼 트랜스포머 인코더 기반이기 때문에, 문장 분류, 문장 관계 판단, 개체명 인식 등 BERT와 동일한 구조로 미세 조정 가능

T5

- T5 (Text-to-Text Transfer Transformer) : 2019년 Google이 발표한 트랜스포머 기반 자연어 처리 모델로 Text-to-Text 구조를 채택 = 입력과 출력을 모두 텍스트 시퀀스로 처리
- 구조 및 성능
 - 인코더-디코더 구조 기반
 - GLUE, SuperGLUE, CNN/DM 등의 데이터셋에서 SOTA(State-of-the-art) 달성
 - 번역, 요약, 질문응답 등 다양한 작업에서 우수한 성능
- 기존 모델과의 차이점

기존 모델	T5
입력 → 벡터	입력 자체를 텍스트 시퀀스로 처리
출력 → 클래스 or 값	출력도 텍스트 시퀀스
구조 유연성 낮음	유연하고 확장성 높음

 - 기존 NLP 모델은 입력을 벡터로 변환 후 분류/회귀 등의 형태로 출력
 - T5는 입력과 출력 모두를 텍스트로 간주하여 처리
 - 다양한 작업을 하나의 통일된 프레임워크로 수행 가능
- 다양한 응용 작업
 - 문장 번역 / 텍스트 요약 / 질의 응답 / 문장 유사도 측정 (STS-B) / 언어 적합성 판단 (CoLA)

사전 학습과 미세 조정

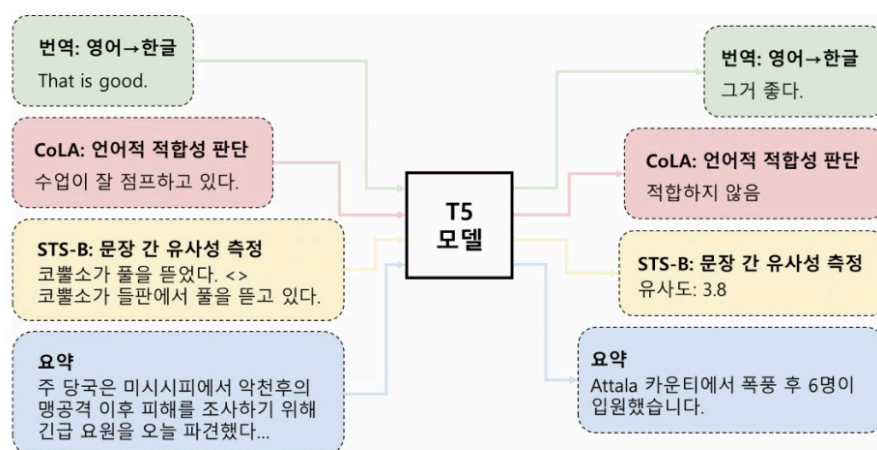


그림 7.21 T5 모델 학습 유형

- 입력과 출력이 모두 텍스트이므로, 태스크 간 통일된 학습과 추론 가능 → 사전 학습 후 미세 조정 단계에서 해당 작업의 데이터를 이용해 모델을 조정해 최적의 성능을 얻을 수 있음

1. 사전 학습

- C4(Cleaned Colossal Common Crawl) 등 방대한 코퍼스를 기반
- 비지도 학습 방식: 문장에서 특정 스펠(span)을 마스킹
- 마스킹된 스펠을 예측하는 방식으로 학습

2. 센티넬 토큰 (Sentinel Token) : 문장마다 유일한 마스크 토큰

- 마스킹된 구간은 `[extra_id_0]` , `[extra_id_1]` ... 식으로 표기
- 입력: `[extra_id_0]`, 인코더, `[extra_id_1]`, 디코더
- 출력: `[extra_id_0]`, 디코더, `[extra_id_1]`
- 총 100개의 sentinel 토큰 사용 가능

미세 조정 방식

- 지도 학습 방식으로 작업별로 추가 fine-tuning → 번역, 문서 요약, 질의응답 등 다양한 다운스트림 태스크에 적용
- 예시 입력

`translate English to German: That is good.`

`summarize: The US president gave a speech...`

- 작업명을 문장 앞에 명시하여 task-specific instruction을 함께 학습